

Week 2 Module

1. Linear Search

- also known as sequential search
- simplest searching algorithm
- it checks each element one by one starting at the very beginning of a list or array until the desired element is found or the end of the list or array is reached.

Logic

Step 1:- Starts from the 0th index.

Step 2:- Compare the current element with the target value.

Step 3:- if the element matches the target, return its index (search successful).

Step 4:- if it doesn't match, move to the next index.

Step 5:- repeat steps 2-4 until the array ends.
If the array ends and the element hasn't found, return -1 (search unsuccessful).

1. Find Student Roll Number (Linear Search)

Problem statement:- Your class teacher has a list of students roll numbers stored in the order they registered. Since the list is unordered, you need to find whether a particular student roll number exists.

WAP to search for the given roll number using linear search.

Input Format:-

- First line contains integer N (no. of students)
- Second line contains N roll numbers
- Third line contains integer x (roll number to search)

Output Format:-

- Print "Found at index i " if present
- otherwise print "Not Found"

Constraints:-

- $1 \leq N \leq 1000$

Example:-

Input

5

101 105 113 110 130

110

Output

Found at index 3


```
#include <stdio.h>
```

```
int main()  
{
```

```
    int N, x;
```

```
    int roll[1000];
```

```
    printf("Enter number of students : ");
```

```
    scanf("%d", &N);
```

```
    printf("Enter roll numbers : ");
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        scanf("%d", &roll[i]);
```

```
    }
```

```
    printf("Enter roll number to be searched : ");
```

```
    scanf("%d", &x);
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        if(roll[i] == x)
```

```
        {
```

```
            printf("Found at index %d\n", i);
```

```
            return 0;
```

```
        }
```

```
    }
```

```
    printf("Not Found\n");
```

```
    return 0;
```

```
}
```

main.c



Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     int N, X;
5     int roll[1000];
6
7     printf("Enter number of students : ");
8     scanf("%d", &N);
9
10    printf("Enter roll numbers : ");
11    for (int i = 0; i < N; i++) {
12        scanf("%d", &roll[i]);
13    }
14
15    printf("Enter roll number to be searched : ");
16    scanf("%d", &X);
17
18    for (int i = 0; i < N; i++) {
19        if (roll[i] == X) {
20            printf("Found at index %d\n", i);
21            return 0;
22        }
23    }
24
25    printf("Not found\n");
26
27    return 0;
28 }
```

```
Enter number of students : 5
Enter roll numbers : 101
105
113
110
130
Enter roll number to be searched : 110
Found at index 3
```

=== Code Execution Successful ===

2. Binary search

- used to find a target value within a sorted array.
- it uses "divide and conquer" strategy to repeatedly halve the search space.
- it compares the target value to the middle element of the array.

Logic

step 1:- set two pointers: low at index 0 and high at the end of the array.

step 2:- Find the middle element:
$$\text{mid} = \text{low} + \frac{\text{high} - \text{low}}{2}$$

step 3:- if the middle element matches the target, return its index

step 4:- if the target is smaller than the middle element, narrow the search to the left half ($\text{high} = \text{mid} - 1$).

step 5:- if the target is greater than the middle element, narrow the search to the right ($\text{low} = \text{mid} + 1$).

step 6:- repeat steps 2-5 until low exceeds high.
If this happens, the element is not present in the array.

2. Search Product ID (Binary search)

Problem Statement :-

An online store keeps product IDs sorted in ascending order. A customer wants to know whether a particular product exists in inventory.

WAP to search the product using Binary search

Input Format :-

- First line contains N
- Second line contains sorted product IDs
- Third line contains target ID

Output :-

Print :

Product Available

OR

Product Not Available

Constraints :-

- $1 \leq N \leq 10^5$

Example :-

Input

6

10 20 30 40 50 60

40

Output

Product Available


```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int N, target;
```

```
    printf("Enter number of product IDs:");
```

```
    scanf("%d", &N);
```

```
    int arr[N];
```

```
    printf("Enter sorted product IDs:");
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    printf("Enter target ID:");
```

```
    scanf("%d", &target);
```

```
    int low=0, high=N-1;
```

```
    int found=0;
```

```
    while(low <= high)
```

```
    {
```

```
        int mid = low + (high - low) / 2;
```

```
        if(arr[mid] == target)
```

```
        {
```

```
            found = 1;
```

```
            break;
```

```
        } else if(arr[mid] < target) {
```

```
            low = mid + 1;
```

```
        } else {
```

```
            high = mid - 1;
```

```
        }
```

```
    }
```

```
    if(found)
```

```
        printf("Product available\n");
```

```
    else
```

```
printf("Product Not available\n");
```

```
return 0;
```

```
}
```


main.c



Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     int N, target;
5
6     printf("Enter number of product IDs : ");
7     scanf("%d", &N);
8     int arr[N];
9
10    printf("Enter sorted product IDs : ");
11    for (int i = 0; i < N; i++) {
12        scanf("%d", &arr[i]);
13    }
14
15    printf("Enter target ID : ");
16    scanf("%d", &target);
17
18    int low = 0, high = N - 1;
19    int found = 0;
20
21    while (low <= high) {
22        int mid = low + (high - low) / 2;
23
24        if (arr[mid] == target) {
25            found = 1;
26            break;
27        }
28        else if (arr[mid] < target) {
29            low = mid + 1;
30        }
31        else {
32            high = mid - 1;
33        }
34    }
35
36    if (found)
37        printf("Product Available\n");
38    else
39        printf("Product Not Available\n");
40
41    return 0;
42 }
```

```
Enter number of product IDs : 6
Enter sorted product IDs : 10
20
30
40
50
60
Enter target ID : 40
Product Available
```

```
=== Code Execution Successful ===
```

3. Bubble Sort

- one of the simplest sorting algorithm.
- it compares adjacent element, and swap them if they are in the wrong order.
- this process is repeated until the entire array is sorted.

Logic

Step 1:- starts at index 0.

Step 2:- compare the current element with next element ($arr[i]$ and $arr[i+1]$).

Step 3:- If the current element is greater than the next element, swap them.

Step 4:- Move to the next pair of elements and repeat steps 2-3.

Step 5:- By the end of the first pass, the largest element will have "bubbled up" to its correct position at the end of the array.

Step 6:- Repeat the entire process for the remaining elements until no more swaps are needed.

3. Arrange Exam score (Bubble sort)

Problem statement:-

A teacher wants to arrange student's exam score in ascending order to prepare the result sheet.

WAP to using Bubble sort.

Input Format:-

- First line contains N
- Second line contains N integers

Output Format:- Print the sorted array.

Example:-

Input

5

89 45 78 12 99

Output

12 45 78 89 99

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int N;
```

```
    printf("Enter number of students : ");
```

```
    scanf("%d", &N);
```

```
    int scores[N];
```

```
    printf("Enter scores : ");
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        scanf("%d", &scores[i]);
```

```
    }
```

```
    for(int i=0; i<N-1; i++)
```

```
    {
```

```
        for(int j=0; j<N-1-i; j++)
```

```
        {
```

```
            if(scores[j] > scores[j+1])
```

```
            {
```

```
                int temp = scores[j];
```

```
                scores[j] = scores[j+1];
```

```
                scores[j+1] = temp;
```

```
            }
```

```
        }
```

```
    } printf("The sorted scores : ");
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        printf("%d", scores[i]);
```

```
    }
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```




programiz.com/onlin



5



main.c



Run

Output

```
1  #include <stdio.h>
2
3  int main() {
4      int N;
5
6      printf("Enter number of students : ");
7      scanf("%d", &N);
8      int scores[N];
9
10     printf("Enter scores : ");
11     for (int i = 0; i < N; i++) {
12         scanf("%d", &scores[i]);
13     }
14
15     for (int i = 0; i < N - 1; i++) {
16         for (int j = 0; j < N - 1 - i; j++) {
17             if (scores[j] > scores[j + 1]) {
18                 int temp = scores[j];
19                 scores[j] = scores[j + 1];
20                 scores[j + 1] = temp;
21             }
22         }
23     }
24
25     for (int i = 0; i < N; i++) {
26         printf("%d ", scores[i]);
27     }
28     printf("\n");
29
30     return 0;
31 }
```

Enter number of students : 5

Enter scores : 89

45

78

12

99

12 45 78 89 99

=== Code Execution Successful ===

4. Insertion sort

- is a simple, comparison-based sorting algorithm that builds a sorted array one element at a time.
- the algorithm conceptually splits the array into two sections: a sorted part and an unsorted part.
- elements are sequentially picked from the unsorted side and "inserted" into their exact correct position within the sorted side.
- sorted part on the left and unsorted part on the right.

Logic

- Step 1:- Assume the first element at index 0 is already sorted.
- Step 2:- Pick the next element from the unsorted part. This element is called the key.
- Step 3:- Compare the key with the elements in the sorted part, moving from right to left.
- Step 4:- Shift all elements in the sorted part that are greater than the key one position to the right to make a vacancy.
- Step 5:- Insert the key into that vacant slot.
- Step 6:- Repeat steps 2-5 until the entire array is sorted.

4. Arrange Library Book Numbers (Insertion sort)

Problem statement:-

A librarian receives book serial numbers one by one and wants to place each new book in the correct position.

WAP to using Insertion sort to arrange book numbers.

Input Format:-

- First line contains N
- Second line contains N integers

Output Format:- Print sorted book numbers

Examples:-

Input

5

55 23 87 11 34

Output

11 23 34 55 87

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int N;
```

```
    printf("Enter number of books:");
```

```
    scanf("%d", &N);
```

```
    int books[N];
```

```
    printf("Enter book numbers:");
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        scanf("%d", &books[i]);
```

```
    }
```

```
    for(int i=1; i<N; i++)
```

```
    {
```

```
        int key = books[i];
```

```
        int j = i-1;
```

```
        while (j >= 0 && books[j] > key)
```

```
        {
```

```
            books[j+1] = books[j];
```

```
            j--;
```

```
        }
```

```
        books[j+1] = key;
```

```
    }
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        printf("%d", books[i]);
```

```
    }
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```




programiz.com/onlin



5



main.c



Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     int N;
5     printf("Enter number of books : ");
6     scanf("%d", &N);
7
8     int books[N];
9     printf("Enter book numbers : ");
10    for (int i = 0; i < N; i++) {
11        scanf("%d", &books[i]);
12    }
13
14    for (int i = 1; i < N; i++) {
15        int key = books[i];
16        int j = i - 1;
17
18        while (j >= 0 && books[j] > key) {
19            books[j + 1] = books[j];
20            j--;
21        }
22        books[j + 1] = key;
23    }
24
25    printf("Sorted book numbers : ");
26    for (int i = 0; i < N; i++) {
27        printf("%d ", books[i]);
28    }
29    printf("\n");
30
31    return 0;
32 }
```

Enter number of books : 5
Enter book numbers : 55
23
87
11
34
Sorted book numbers : 11 23 34 55 87

=== Code Execution Successful ===

5. Selection Sort

- it divides the array into a sorted part (on the left) and an unsorted part (on the right).
- it finds the smallest or largest element from the unsorted portion of the array and swaps it with the first element of that unsorted portion.

Logic

- Step 1:- Start at index 0. Assume this is the minimum element.
- Step 2:- Scan the rest of the unsorted array to find the actual minimum value.
- Step 3:- Once found, swap the actual minimum value with the element at the starting position.
- Step 4:- Move to the boundary of the sorted array to one step to the right.
- Step 5:- Repeat the process for the remaining unsorted elements until the entire array is sorted.

5. Rank Players by score (selection sort)

Problem statement:-

A gaming club wants to rank players based on scores from lowest to highest.

WAP using selection sort.

Input Format:-

- First line contains N
- Second line contains player scores

Output Format:- Print sorted scores.

Examples:-

Input

5

70 20 90 10 40

Output

10 20 40 70 90

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int N;
```

```
    printf("Enter number of players:");
```

```
    scanf("%d", &N);
```

```
    int scores[N];
```

```
    printf("Enter scores:");
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        scanf("%d", &scores[i]);
```

```
    }
```

```
    for(int i=0; i<N-1; i++)
```

```
    {
```

```
        int min_idx = i;
```

```
        for(int j=i+1; j<N; j++)
```

```
        {
```

```
            if(scores[j] < scores[min_idx]) {
```

```
                min_idx = j;
```

```
            }
```

```
        }
```

```
        int temp = scores[i];
```

```
        scores[i] = scores[min_idx];
```

```
        scores[min_idx] = temp;
```

```
    }
```

```
    for(int i=0; i<N; i++)
```

```
    {
```

```
        printf("%d", scores[i]);
```

```
    }
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```




main.c



Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     int N;
5     printf("Enter number of players : ");
6     scanf("%d", &N);
7
8     int scores[N];
9     printf("Enter scores : ");
10    for (int i = 0; i < N; i++) {
11        scanf("%d", &scores[i]);
12    }
13
14    for (int i = 0; i < N - 1; i++) {
15        int min_idx = i;
16        for (int j = i + 1; j < N; j++) {
17            if (scores[j] < scores[min_idx]) {
18                min_idx = j;
19            }
20        }
21
22        int temp = scores[i];
23        scores[i] = scores[min_idx];
24        scores[min_idx] = temp;
25    }
26
27    printf("sorted scores : ");
28    for (int i = 0; i < N; i++) {
29        printf("%d ", scores[i]);
30    }
31    printf("\n");
32
33    return 0;
34 }
```

```
Enter number of players : 5
Enter scores : 70
20
90
10
40
sorted scores : 10 20 40 70 90
```

```
=== Code Execution Successful ===
```